

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

La primera universidad agropecuaria del Ecuador

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

CARRERA DE SOFTWARE (REDISEÑO)

PRÁCTICA EXPERIMENTAL U3

Clúster Distribuido de Base de Datos con CockroachDB: Fragmentación, Tolerancia a Fallos y Análisis Comparativo de Rendimiento en el Sistema TiendaTech

Asignatura:	Aplicaciones Distribuidas [20701]
Nivel / Paralelo:	7mo Nivel – A
Profesor:	Guerrero Ulloa Gleiston Cicerón
Periodo académico:	2026-2027 PPA

Grupo C

Jeremy Gaibor
Freddy Farinango – 1050453776
Iván Villamarín

Quevedo – Los Ríos, Ecuador
2026

Resumen

Objetivo: Implementar, configurar y verificar un clúster distribuido de tres nodos con CockroachDB v24.3.0 sobre contenedores Docker, aplicando la estrategia de fragmentación horizontal por lista del sistema TiendaTech, con el fin de demostrar tolerancia a fallos mediante consenso Raft y comparar cuantitativamente su rendimiento frente a una instancia de nodo único. **Método:** Se desplegó un clúster de tres nodos (`crdb-node1..3`) en red Docker *bridge*, se implementó el esquema relacional de TiendaTech (cuatro tablas con claves foráneas y restricciones `CHECK`), se aplicó `PARTITION BY LIST` sobre `provincia_entrega` y se cargaron 10 000 pedidos reproducibles (semilla 42) mediante un script Python con `psycpg2`. Se detuvo un nodo para verificar el quórum Raft (2 de 3 réplicas) y se ejecutaron cinco consultas representativas con `EXPLAIN ANALYZE`, cada una repetida cinco veces, en el clúster y en una instancia `start-single-node` equivalente. **Resultados:** El clúster permaneció disponible y consistente durante la caída del nodo 2 (conteo verificado en 10 000 pedidos antes y después de la caída), y se reintegró en 5.73 segundos. La tabla, aunque lógicamente fragmentada en cinco particiones, residió físicamente en un único rango replicado en los tres nodos, dado que su tamaño (menor a 512 MiB) no alcanzó el umbral de división automática. Los resultados de rendimiento fueron mixtos: el nodo único fue más rápido en tres de las cinco consultas evaluadas (Q1, Q2, Q3), mientras que el clúster fue más rápido en las otras dos (Q4, Q5), con factores de comparación entre 0.321 y 1.520 (promedio 0.949). **Conclusión:** El clúster de tres nodos garantizó disponibilidad ante el fallo de un nodo sin pérdida de datos; su rendimiento, en cambio, no fue uniformemente mejor ni peor que el de un nodo único, sino que dependió del tipo y costo de cada consulta. El principal beneficio observado de CockroachDB en este escenario fue la continuidad del servicio, no una ganancia de rendimiento generalizada.

Abstract: Objective: To deploy, configure, and verify a distributed three-node cluster using CockroachDB v24.3.0 on Docker containers, applying the horizontal list-partitioning strategy of the TiendaTech system, in order to demonstrate fault tolerance through Raft consensus and quantitatively compare its performance against a single-node instance. **Method:** A three-node cluster (`crdb-node1..3`) was deployed on a Docker bridge network; the TiendaTech relational schema (four tables with foreign keys and `CHECK` constraints) was implemented; `PARTITION BY LIST` was applied on `provincia_entrega`; and 10,000 reproducible orders (seed 42) were loaded via a Python/`psycpg2` script. One node was

stopped to verify Raft quorum (2 of 3 replicas), and five representative queries were executed with `EXPLAIN ANALYZE`, each repeated five times, on both the cluster and an equivalent `start-single-node` instance. **Results:** The cluster remained available and consistent while node 2 was down (row count verified at 10,000 both before and after the failure), and reintegrated in 5.73 seconds. Although logically split into five partitions, the table physically resided in a single replicated range across the three nodes, since its size (under 512 MiB) never reached the automatic split threshold. Performance results were mixed: the single node was faster in three of the five evaluated queries (Q1, Q2, Q3), while the cluster was faster in the other two (Q4, Q5), with comparison factors ranging from 0.321 to 1.520 (average 0.949). **Conclusion:** The three-node cluster guaranteed availability under a single-node failure without data loss; its performance, however, was not uniformly better or worse than a single node, but rather depended on the type and cost of each query. CockroachDB's main observed benefit in this scenario was service continuity, not a generalized performance gain.

Contents

1	Introducción	2
1.1	Pregunta de investigación	2
2	Fundamento Teórico	3
2.1	Fundamentos y Arquitecturas de Bases de Datos Distribuidas	3
2.2	Fragmentación y Replicación de Datos . .	3
2.3	Control de Concurrencia Distribuida . . .	4
2.4	Consistencia, Disponibilidad, PACELC y Sistemas NoSQL	4
3	Materiales y Métodos	5
3.1	Entorno, herramientas y control de versiones	5
3.2	Despliegue del clúster (Paso 1)	5
3.3	Esquema de datos de TiendaTech (Paso 2)	5
3.4	Composición del dataset sintético	6
3.5	Protocolo de verificación de tolerancia a fallos (Paso 3)	6
3.6	Consultas de referencia y <i>benchmark</i> (Paso 4)	6
3.7	Registro en video de la prueba de tolerancia a fallos	7
3.8	Comandos de verificación utilizados	7
4	Resultados	7
4.1	Tolerancia a fallos	7
4.2	Fragmentación lógica vs. rangos físicos . .	7
4.3	Rendimiento: clúster vs. nodo único . . .	8

4.4	Análisis del plan de ejecución (EXPLAIN ANALYZE)	9
4.5	Síntesis cuantitativa de resultados	9
5	Discusión	9
5.1	Contraste con trabajos relacionados	9
5.2	Amenazas a la validez	10
5.3	Trabajo futuro	10
6	Conclusiones	10
7	Anexos	11

List of Figures

1	Dashboard de CockroachDB (localhost:8080) mostrando los tres nodos activos del clúster.	5
2	Salida de SHOW PARTITIONS FROM TABLE pedidos: cinco particiones lógicas registradas por provincia.	6
3	Confirmación de la carga de 10 000 pedidos y 19 884 detalles en el clúster.	6
4	Salida de SHOW RANGES FROM TABLE pedidos durante la caída del nodo 2: la configuración de réplicas {1,2,3} no cambió dentro de la ventana de 30 s.	7
6	Reintegración del nodo 2 al clúster: los tres nodos vuelven a reportar is_live=true tras docker start crdb-node2.	7
5	Diagrama de secuencia del comportamiento Raft ante la caída del nodo 2: el clúster mantiene disponibilidad de lectura con quórum 2/3.	8
7	Gráfico comparativo de la latencia promedio (ms) por consulta, clúster de 3 nodos vs. nodo único, generado a partir de evidencia/resultados.csv.	9
8	Plan de ejecución real de Q1 (EXPLAIN ANALYZE) en el clúster: lookup join entre pedidos y clientes.	9
9	Plan de ejecución real de Q3 (EXPLAIN ANALYZE): búsqueda puntual por clave primaria compuesta, actual row count = 1.	9

List of Tables

1	Entorno de ejecución del experimento	5
2	Comandos de verificación ejecutados y evidencia que producen	7
3	Tiempo promedio de ejecución (ms, $n = 5$) y factor de comparación entre nodo único y clúster	8

1. Introducción

La computación distribuida ha dejado de ser un recurso exclusivo de compañías de escala global para convertirse en un requisito habitual de cualquier sistema transaccional que aspire a alta disponibilidad. Los Sistemas de Gestión de Bases de Datos Distribuidas (SGBDD) permiten que los datos residan en múltiples nodos físicos o lógicos, ofreciendo al usuario final la ilusión de un sistema centralizado (transparencia de distribución) mientras internamente resuelven problemas de fragmentación, replicación, concurrencia y recuperación ante fallos [1].

El Proyecto de Formación de Carrera (PFC) del grupo, TiendaTech, es una plataforma de comercio electrónico de hardware que requiere escalar horizontalmente su capa de persistencia y tolerar la caída de nodos individuales sin interrumpir el servicio. CockroachDB fue seleccionado en semanas previas del curso como motor NewSQL objetivo por implementar de manera nativa el algoritmo de consenso Raft, fragmentación lógica configurable y semántica transaccional SERIALIZABLE sin depender de hardware especializado, a diferencia de sistemas NoSQL puramente eventualmente consistentes [2].

Esta práctica experimental cierra el ciclo de diseño—implementación de la Unidad 2 (fragmentación y replicación) y la Unidad 3 (tolerancia a fallos y rendimiento distribuido) del curso de Aplicaciones Distribuidas, llevando a la práctica la estrategia de fragmentación horizontal por provincia diseñada en semanas previas del PFC, y verificando de forma honesta —contrastando cada afirmación con la salida real de los comandos SHOW PARTITIONS y SHOW RANGES— qué ocurre físicamente con esos datos dentro del clúster.

1.1 Pregunta de investigación

Se formulan las siguientes preguntas de investigación (RQ), que guían el diseño experimental y a las que se responde de forma cuantificada y en tiempo pasado en la Sección 6:

- **RQ1.** ¿Fue posible desplegar y verificar, mediante herramientas de contenedores, un clúster CockroachDB de tres nodos que replicara automáticamente los datos del esquema TiendaTech con fragmentación horizontal por `provincia_entrega`?
- **RQ2.** ¿El clúster preservó la disponibilidad y la consistencia de las consultas ante la caída de un nodo, en virtud del quórum Raft (2 de 3 réplicas)?
- **RQ3.** ¿Cuál fue la magnitud del efecto del modo de despliegue (clúster de tres nodos vs. nodo único) sobre la latencia de consultas representativas (*join*, agregación, búsqueda puntual, rango y subconsulta correlacionada) a una escala de 10 000 pedidos?

2. Fundamento Teórico

2.1 Fundamentos y Arquitecturas de Bases de Datos Distribuidas

Una Base de Datos Distribuida (BDD) se define formalmente como una colección de múltiples bases de datos lógicamente interrelacionadas, distribuidas sobre una red de computadoras, y gestionadas por un SGBDD que hace que dicha distribución sea transparente para el usuario [1]. Si R es una relación global, un SGBDD debe garantizar que el conjunto de fragmentos $\{R_1, R_2, \dots, R_n\}$ almacenados en los nodos $\{S_1, S_2, \dots, S_n\}$ pueda reconstruirse en R mediante un operador de reconstrucción ρ , de modo que $R = \rho(R_1, R_2, \dots, R_n)$, sin que el usuario deba conocer la ubicación física de cada fragmento.

Date propuso doce objetivos de diseño que caracterizan a un SGBDD ideal: (1) autonomía local de cada nodo, (2) no dependencia de un sitio central, (3) operación continua, (4) independencia de la ubicación, (5) independencia de la fragmentación, (6) independencia de la replicación, (7) procesamiento distribuido de consultas, (8) gestión distribuida de transacciones, (9) independencia del hardware, (10) independencia del sistema operativo, (11) independencia de la red y (12) independencia del SGBD subyacente (heterogeneidad) [1]. CockroachDB satisface parcialmente estos objetivos: cumple con (1)-(8) al ofrecer autonomía de rangos, ausencia de nodo maestro fijo (elección dinámica de líder Raft por rango) y balanceo automático, pero no persigue (12), ya que es un sistema homogéneo en el que todos los nodos ejecutan el mismo motor.

En cuanto a arquitecturas de SGBD distribuidos, la literatura distingue tres modelos: cliente-servidor, donde los clientes envían solicitudes SQL a un servidor que resuelve la consulta sin que el cliente conozca la distribución interna; colaborativa (o *peer-to-peer*), donde cada nodo actúa simultáneamente como cliente y servidor y coopera para resolver consultas multinodo —modelo que sigue CockroachDB al enrutar internamente cada solicitud SQL hacia el nodo que posee la réplica líder del rango correspondiente—; y *middleware*, donde una capa intermedia oculta múltiples bases de datos heterogéneas subyacentes [1].

La transparencia de distribución se articula típicamente en ocho formas: de fragmentación, de ubicación, de replicación, de concurrencia, de fallos, de diseño, de ejecución y de escalado [1]. En la práctica realizada, la transparencia de fragmentación y de fallos se evidenció directamente: la aplicación cliente (`seed_data.py`, `run_benchmark.py`) ejecutó las mismas sentencias SQL contra `tiendatech` sin necesitar saber, para operar, que `pedidos` estaba particionada lógicamente en cinco fragmentos ni que dicha tabla estaba replicada en los tres nodos del clúster —aunque, como se detalla en la Sección 4.2, esa fragmentación lógica no implicó una distribución física en múltiples rangos a la escala de esta práctica—.

Aplicación concreta a TiendaTech: de los doce objetivos de Date, dos se verificaron empíricamente en esta práctica y no solo de forma teórica. La “operación continua” (objetivo 3) se comprobó en la Sección 4.1, donde el clúster continuó respondiendo consultas durante la caída simulada del nodo 2. La “independencia de la fragmentación” (objetivo 5) se comprobó en que ni `seed_data.py` ni `run_benchmark.py` necesitaron conocer, para funcionar correctamente, si la tabla `pedidos` estaba fragmentada en 1 o en 5 particiones lógicas —cuestión que, como se documenta en la Sección 4.2, resultó ser irrelevante a nivel físico en esta escala de datos—. Los objetivos 9-11 (independencia de hardware, sistema operativo y red) se cumplieron trivialmente al ejecutarse los tres nodos como contenedores Docker idénticos sobre el mismo host.

Finalmente, se distingue entre sistemas homogéneos, donde todos los nodos ejecutan idéntico SGBD y modelo de datos (caso de nuestro clúster de tres instancias CockroachDB v24.3.0), y heterogéneos, donde coexisten SGBD distintos integrados mediante *wrappers* o *middleware* de federación. La homogeneidad simplifica drásticamente el diseño porque evita la traducción de esquemas y de semántica transaccional entre motores distintos, a costa de menor flexibilidad tecnológica.

2.2 Fragmentación y Replicación de Datos

La fragmentación consiste en dividir una relación global R en subconjuntos $R_1, \dots, R_n$ que se almacenan en distintos nodos. Para que la fragmentación sea correcta debe cumplir tres condiciones formales [1]: **completitud** —todo elemento de R debe pertenecer a algún fragmento, $R = \bigcup_{i=1}^n R_i$ —; **reconstrucción** —debe existir un operador relacional ρ tal que $R = \rho(R_1, \dots, R_n)$ —; y **disjunción** —para fragmentación horizontal, $R_i \cap R_j = \emptyset$ para todo $i \neq j$ —.

La **fragmentación horizontal** divide una relación mediante un predicado de selección:

$$R_i = \sigma_{p_i}(R), \quad i = 1, \dots, m, \quad (1)$$

donde σ es el operador de selección. En TiendaTech, la tabla `pedidos` se fragmentó lógicamente sobre el atributo `provincia_entrega` mediante `PARTITION BY LIST`, con

$$\begin{aligned} p_i : \text{provincia_entrega} &= v_i, \\ v_i &\in \{\text{L.RÍOS, GUAYAS, PICHINCHA,} \\ &\quad \text{MANABÍ, AZUAY}\}, \end{aligned} \quad (2)$$

de modo que $R_{\text{pedidos}} = \sigma_{v_1}(R) \cup \dots \cup \sigma_{v_5}(R)$, satisfaciendo completitud y disjunción. Se eligió `PARTITION BY LIST` en lugar de `PARTITION BY RANGE` porque el dominio de provincias es un conjunto pequeño, discreto y conocido de antemano.

Precisión conceptual importante: en CockroachDB, `PARTITION BY LIST` define particiones lógicas —unidades de configuración de zona (*zone config*)— pero

no garantiza por sí sola que cada partición ocupe un rango físico distinto. Un rango se subdivide automáticamente solo cuando su tamaño supera el umbral configurado (por defecto 512 MiB) o cuando existe una directiva explícita de división. La Sección 4.2 contrasta esta definición formal con la salida real de `SHOW PARTITIONS` y `SHOW RANGES` obtenida en la práctica.

La **fragmentación vertical** divide R por columnas: $R_i = \pi_{A_i \cup K}(R)$, donde π es la proyección, A_i es un subconjunto de atributos y K es la clave primaria. La decisión de qué atributos agrupar se basa en un análisis de *afinidad* [1]. La **fragmentación mixta** combina ambas estrategias.

En cuanto a **replicación**, un sistema puede mantener copias *síncronas* o *asíncronas* de cada fragmento. En replicación síncrona, una escritura no se confirma al cliente hasta que un quórum de réplicas la ha persistido; es el modelo que usa CockroachDB mediante Raft, donde cada rango requiere que $\lceil (n+1)/2 \rceil$ de n réplicas confirmen la entrada del *log* antes de aplicarla [2]. En replicación asíncrona, el nodo primario confirma la escritura de inmediato y propaga los cambios en segundo plano, exponiendo una ventana de inconsistencia. Los resultados de la Sección 4 muestran que este costo de latencia se manifestó de forma desigual entre consultas: dominó en las operaciones más simples (Q1-Q3), pero no impidió que el clúster fuera más rápido en las de mayor costo de cómputo (Q4-Q5).

2.3 Control de Concurrency Distribuida

El control de concurrencia en un entorno distribuido debe garantizar el aislamiento de transacciones que se ejecutan simultáneamente sobre fragmentos ubicados en distintos nodos, un problema más complejo que en un SGBD centralizado porque no existe un único punto de sincronización ni un reloj global compartido [1].

El protocolo de **bloqueo en dos fases (2PL)** distribuido extiende el 2PL centralizado exigiendo que cada nodo que participa en una transacción adquiera bloqueos sobre los datos locales que accede, respetando una fase de crecimiento seguida de una fase de decrecimiento. La variante **2PL estricto** retiene todos los bloqueos de escritura hasta el `COMMIT` o `ABORT`.

Ejemplo aplicado a TiendaTech: considérese dos transacciones concurrentes T_1 y T_2 que procesan pedidos distintos pero comparten el mismo `producto_id`: T_1 actualiza `productos.stock` tras confirmar un pedido en GUAYAS mientras T_2 hace lo propio para un pedido en MANABÍ. Bajo 2PL estricto, T_1 adquiere un bloqueo de escritura sobre la fila de `productos` y lo retiene hasta su `commit`; T_2 , que solicita el mismo bloqueo, debe esperar. Si además se formara un ciclo de espera cruzado, se produciría un interbloqueo distribuido que el gestor de bloqueos debe detectar y resolver abortando una de las dos transacciones. CockroachDB resuelve este caso mediante detección de conflictos basada en *timestamps*

y reintento automático de la transacción más joven.

Cuando distintos nodos mantienen bloqueos solicitados de forma cruzada, puede surgir un **interbloqueo distribuido** (*deadlock*), detectable mediante un *grafo de espera* global (*wait-for graph*): existe interbloqueo si y solo si el grafo contiene un ciclo.

Para garantizar atomicidad en transacciones multinodo, el protocolo clásico es el **compromiso en dos fases (2PC)**, cuyo problema fundamental es que **bloquea** si el coordinador falla después de que los participantes votaron **SÍ** pero antes de recibir la decisión final [3]. El **compromiso en tres fases (3PC)** intenta resolver este bloqueo, aunque bajo el supuesto de ausencia de particiones de red simultáneas a la falla del coordinador —supuesto que el resultado de imposibilidad de Fischer, Lynch y Paterson (FLP) demuestra que no puede garantizarse en general en un sistema asíncrono con fallos [4]—. CockroachDB evita el bloqueo del 2PC clásico reemplazándolo por un protocolo de consenso Raft por rango, de modo que el propio registro del estado transaccional está replicado y sobrevive a la caída de un nodo individual [2, 3].

Síntesis crítica: Li, Liu y Li muestran que, bajo el diseño de líder fuerte de Raft, todas las escrituras de *log* deben pasar por un único líder, lo que limita el *throughput* cuando el volumen de solicitudes crece; su propuesta de procesamiento por lotes asíncrono (Sección 5) es directamente relevante para explicar la penalización de latencia observada en las consultas más simples de la Sección 4 de esta práctica (Q1-Q3), aunque dicha penalización no se mantuvo en las consultas de mayor costo (Q4-Q5) [5].

2.4 Consistencia, Disponibilidad, PACELC y Sistemas NoSQL

El **teorema CAP** establece que un sistema distribuido no puede garantizar simultáneamente Consistencia, Disponibilidad y Tolerancia a Particiones; ante una partición de red, el sistema debe elegir entre sacrificar consistencia (modelo AP) o disponibilidad (modelo CP) [6]. La interpretación moderna del teorema matiza que la elección no es binaria ni permanente [6].

Abadi propuso el modelo **PACELC**: si hay Partición (P), el sistema elige entre Disponibilidad (A) y Consistencia (C); en caso contrario (*Else*, E), elige entre Latencia (L) y Consistencia (C) [7]. CockroachDB se clasifica como PC/EC: ante partición prioriza consistencia y, en ausencia de partición, también prioriza consistencia fuerte sobre la latencia mínima posible, lo que explica en parte la penalización de latencia observada en las consultas más simples de nuestros experimentos frente al nodo único (Sección 4), aunque no la ventaja del clúster en las consultas de mayor costo computacional.

La **consistencia eventual** garantiza únicamente que, en ausencia de nuevas escrituras, todas las réplicas convergerán finalmente al mismo valor; es el modelo por

defecto de sistemas como Apache Cassandra (AP/EL). La **consistencia fuerte serializable** de CockroachDB garantiza que el resultado de cualquier conjunto de transacciones concurrentes es equivalente a alguna ejecución serial de las mismas [2]. Un enfoque intermedio influyente es Google Spanner, que logra consistencia externa global mediante TrueTime [2, 8].

Respecto a sistemas NewSQL comparables, TiDB adopta igualmente almacenamiento distribuido basado en Raft (TiKV) separado de una capa SQL sin estado [9]. Pina, Sá y Bernardino evaluaron experimentalmente CockroachDB junto con MariaDB Xpand y VoltDB, encontrando que CockroachDB presentó las latencias más altas entre las tres, aunque con mejor escalabilidad al aumentar el volumen de datos [10]. Este hallazgo —de un estudio Scopus-indexado y publicado en 2023— es parcialmente coherente con el patrón observado en nuestra práctica: a la escala reducida de 10 000 pedidos, la latencia de CockroachDB fue mayor que la de un nodo único en las consultas más simples, aunque no en todas las consultas evaluadas; se retoma esta comparación con mayor detalle en la Sección 5.

Relación directa con TiendaTech: dado que el dominio transaccional de TiendaTech exige integridad referencial estricta y prevención de condiciones de carrera sobre el inventario, un modelo de consistencia eventual expondría al sistema al riesgo de vender el mismo producto con *stock* agotado a dos clientes simultáneos; la elección de un motor PC/EC como CockroachDB —pese a la penalización de latencia que mostró en las consultas más simples— es la decisión de diseño correcta para esa clase de riesgo de negocio (Anexo 7).

3. Materiales y Métodos

3.1 Entorno, herramientas y control de versiones

La Tabla 1 resume el entorno de ejecución empleado.

Table 1: Entorno de ejecución del experimento

Componente	Versión / detalle
Sistema operativo host	Windows 10/11 (PowerShell)
Motor de contenedores	Docker Desktop 4.x, Docker Compose Plugin
Imagen CockroachDB	cockroachdb/ cockroach:v24.3.0
Lenguaje de carga de datos	Python 3.12
Librerías Python	psycpg2-binary 2.9.12, faker 40.28.1
Recursos por nodo	mem_limit: 512m; cache: .25; max-sql-mem: .25

Repositorio del proyecto: <https://github.com/JeremyGaibor/pe-u3-crdp-equipo-c>

Rama de entrega: main

Tag de versión: v1.0-pe-u3

Hash del commit final: 460cad0

El código fuente, los scripts SQL, la configuración Docker y las evidencias de esta práctica corresponden exactamente al estado del repositorio en el commit 460cad0 de la rama main, etiquetado como v1.0-pe-u3.

3.2 Despliegue del clúster (Paso 1)

Se definió `docker-compose.yml` con tres servicios (`crdb-node1..3`) sobre la red interna `bridge` `roach-net`, cada uno con puertos SQL/HTTP distintos (26257/8080, 26258/8081, 26259/8082) y `healthcheck` basado en `cockroach node status`. La configuración de cada nodo incluyó `-insecure`, `-cache=.25`, `-max-sql-memory=.25` y `-join` apuntando a los tres nodos.

Tras levantar los contenedores con `docker compose up -d`, se inicializó el clúster una única vez con `cockroach init -insecure` y se verificó el estado de los tres nodos con `cockroach node status`, confirmando `is_live=true` para los identificadores 1, 2 y 3 (Fig. 1).

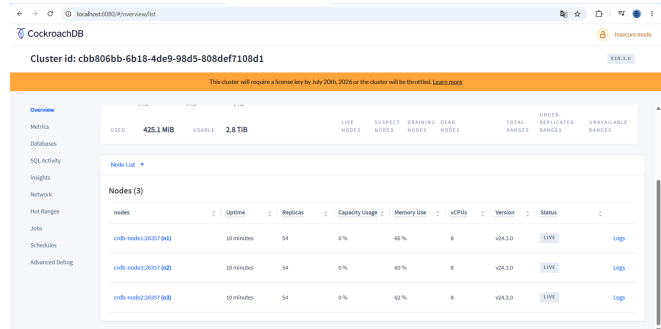


Figure 1: Dashboard de CockroachDB (localhost:8080) mostrando los tres nodos activos del clúster.

3.3 Esquema de datos de TiendaTech (Paso 2)

El esquema (`sql/01_schema.sql`) define cuatro tablas relacionadas: `clientes`, `productos`, `pedidos` y `detalle_pedido`, con claves primarias, foráneas y restricciones CHECK de dominio. La tabla `pedidos` usa una clave primaria compuesta (`provincia_entrega`, `pedido_id`), condición necesaria para que la fragmentación lógica por lista coincida con el prefijo de la clave primaria.

Tras crear el esquema, se aplicó la fragmentación lógica (`sql/02_partitions.sql`): `ALTER TABLE pedidos PARTITION BY LIST (provincia_entrega) (...)`, definiendo cinco particiones. Se verificó con `SHOW PARTITIONS FROM TABLE pedidos` que las cinco particiones quedaron correctamente registradas en el catálogo

(Fig. 2).

database_name	table_name	partition_name	parent_partition	column_names	index_name	partition_value	row_config	full_row_config
tiendatech	pedidos	pedidos_azuay	NUL	provincia,entrega	pedidospk_pedido	('AZUAY')	NUL	range.min.bytes = 134217728, range.max.bytes = 53689912, g.ttl.seconds = 14400, max.replicas = 1, constraints = [], base.preferences = [], range.min.bytes = 134217728, range.max.bytes = 53689912, g.ttl.seconds = 14400, max.replicas = 1, constraints = [], base.preferences = []
tiendatech	pedidos	pedidos_guayas	NUL	provincia,entrega	pedidospk_pedido	('GUAYAS')	NUL	range.min.bytes = 134217728, range.max.bytes = 53689912, g.ttl.seconds = 14400, max.replicas = 1, constraints = [], base.preferences = [], range.min.bytes = 134217728, range.max.bytes = 53689912, g.ttl.seconds = 14400, max.replicas = 1, constraints = [], base.preferences = []
tiendatech	pedidos	pedidos_los_rios	NUL	provincia,entrega	pedidospk_pedido	('LOS RÍOS')	NUL	range.min.bytes = 134217728, range.max.bytes = 53689912, g.ttl.seconds = 14400, max.replicas = 1, constraints = [], base.preferences = [], range.min.bytes = 134217728, range.max.bytes = 53689912, g.ttl.seconds = 14400, max.replicas = 1, constraints = [], base.preferences = []
tiendatech	pedidos	pedidos_manabi	NUL	provincia,entrega	pedidospk_pedido	('MANABÍ')	NUL	range.min.bytes = 134217728, range.max.bytes = 53689912, g.ttl.seconds = 14400, max.replicas = 1, constraints = [], base.preferences = [], range.min.bytes = 134217728, range.max.bytes = 53689912, g.ttl.seconds = 14400, max.replicas = 1, constraints = [], base.preferences = []
tiendatech	pedidos	pedidos_pichincha	NUL	provincia,entrega	pedidospk_pedido	('PICHINCHA')	NUL	range.min.bytes = 134217728, range.max.bytes = 53689912, g.ttl.seconds = 14400, max.replicas = 1, constraints = [], base.preferences = [], range.min.bytes = 134217728, range.max.bytes = 53689912, g.ttl.seconds = 14400, max.replicas = 1, constraints = [], base.preferences = []

Figure 2: Salida de `SHOW PARTITIONS FROM TABLE pedidos`: cinco particiones lógicas registradas por provincia.

Se cargaron los datos con `scripts/seed_data.py`, un script Python reproducible (semillas `random.seed(42)` y `Faker.seed(42)`) que genera y carga, mediante `psycpg2.extras.execute_values` en lotes de 500-1000 filas: 1000 clientes, 500 productos, 10000 pedidos y 19884 líneas de detalle. El dataset es sintético, generado localmente con `Faker (locale es_ES)`. La reproducibilidad se verificó con `SELECT COUNT(*)` sobre cada tabla (Fig. 3).

```

root@crdb-node1:26257/tiendatech> SELECT COUNT(*) AS clientes FROM clientes;
--> SELECT COUNT(*) AS productos FROM productos;
--> SELECT COUNT(*) AS pedidos FROM pedidos;
--> SELECT COUNT(*) AS detalles FROM detalle_pedido;
-->

clientes
-----
1000
(1 row)

Time: 36ms total (execution 36ms / network 0ms)

productos
-----
500
(1 row)

Time: 10ms total (execution 9ms / network 1ms)

pedidos
-----
10000
(1 row)

Time: 90ms total (execution 89ms / network 1ms)

detalles
-----
19884
(1 row)

```

Figure 3: Confirmación de la carga de 10000 pedidos y 19884 detalles en el clúster.

3.4 Composición del dataset sintético

El generador `seed_data.py` construye el dataset a partir de constantes explícitas: `TOTAL_CLIENTES = 1000`, `TOTAL_PRODUCTOS = 500`, `TOTAL_PEDIDOS = 10000`. Las provincias del dominio son LOS RÍOS, GUAYAS, PICHINCHA, MANABÍ y AZUAY, asignadas mediante `random.choice`. Los productos se agrupan en seis categorías: PROCESADORES, TARJETAS GRÁFICAS, MEMORIAS RAM, ALMACENAMIENTO, PLACAS MADRE y PERIFÉRICOS. El estado de cada pedido se sortea entre PENDIENTE, PAGADO, ENVIADO, ENTREGADO y CANCELADO.

Las fechas se generan con `fecha_inicial = datetime.now() - timedelta(days=730)` y un desplazamiento aleatorio de hasta 729 días. Cada pedido incluye entre 1 y 3 líneas de detalle, lo que explica que 10000 pedidos produjeran 19884 filas en `detalle_pedido` —un promedio de 1.99 líneas por pedido—, cifra verificada directamente y no estimada.

3.5 Protocolo de verificación de tolerancia a fallos (Paso 3)

Con el clúster en operación y los 10000 pedidos verificados, se ejecutó el siguiente protocolo:

1. Confirmar el conteo base ejecutando `SELECT COUNT(*) FROM pedidos` contra el nodo 1 (esperado: 10000).
2. Detener el nodo 2 con `docker stop crdb-node2`.
3. Confirmar el estado Exited del nodo 2 mediante `docker compose ps -a`.
4. Reejecutar `SELECT COUNT(*) FROM pedidos` contra el nodo 1, esperando el mismo resultado.
5. Esperar 30 segundos y ejecutar `SHOW RANGES FROM TABLE pedidos` para observar si existió redistribución de réplicas.
6. Reiniciar el nodo 2 con `docker start crdb-node2` y confirmar el retorno a `is_live=true` en los tres nodos.

3.6 Consultas de referencia y benchmark (Paso 4)

Se definieron cinco consultas SQL (`sql/03_queries.sql`): **Q1** `join pedidos x clientes`; **Q2** agregación `GROUP BY` por provincia; **Q3** búsqueda puntual por clave primaria compuesta; **Q4** consulta de rango por fecha y monto; **Q5** subconsulta correlacionada de clientes con más de 5 pedidos. Cada consulta se envolvió en `EXPLAIN ANALYZE`.

El script `scripts/run_benchmark.py` ejecuta cada consulta cinco veces (`REPETICIONES = 5`), extrae el tiempo de ejecución del plan mediante una expresión regular y calcula la media aritmética con `statistics.mean()`.

Limitación reconocida del instrumento: la versión del script utilizada en esta práctica descarta los cinco tiempos individuales tras calcular la media y solo persiste el promedio en `evidencia/resultados.csv`; en consecuencia, no fue posible calcular desviación estándar, mediana, mínimo, máximo ni percentiles sin inventar datos. Los resultados presentados corresponden exclusivamente a los promedios reales obtenidos durante la ejecución del *benchmark*. Esta limitación se reconoce como una amenaza a la validez (Sección 5.2) y como una mejora futura del instrumento (Sección 5.3).

Para la comparación distribuido vs. centralizado se creó `docker-compose-single.yml`, que despliega una única instancia CockroachDB (`crdb-single`, modo `start-single-node`) en el puerto 26260. Sobre esta instancia se replicó exactamente el mismo protocolo, garantizando que ambos entornos contienen datos idénticos y que la única variable independiente es el número de nodos.

3.7 Registro en video de la prueba de tolerancia a fallos

La prueba completa de tolerancia a fallos descrita en la sección anterior (pasos 1 a 6) fue registrada en video y almacenada en el repositorio del proyecto (commit `460cad0`, tag `v1.0-pe-u3`), en la siguiente ruta:

`evidencia/prueba-de-tolerancia-a-fallos.mp4`

3.8 Comandos de verificación utilizados

La Tabla 2 enumera los comandos de verificación efectivamente ejecutados durante la práctica, en el orden en que se usaron.

Table 2: Comandos de verificación ejecutados y evidencia que producen

Comando	Propósito / evidencia
<code>cockroach node status</code>	Confirmar <code>is_live=true</code> en los 3 nodos (Fig. 1, 6).
<code>SHOW PARTITIONS FROM TABLE pedidos;</code>	Verificar las 5 particiones lógicas (Fig. 2).
<code>SELECT COUNT(*) FROM <tabla>;</code>	Verificar filas cargadas (Fig. 3).
<code>SHOW RANGES FROM TABLE pedidos;</code>	Verificar distribución física real (Fig. 4).
<code>docker stop / start crdb-node2</code>	Simular y revertir la caída del nodo 2.
<code>EXPLAIN ANALYZE <consulta>;</code>	Obtener tiempo real y plan físico.

4. Resultados

4.1 Tolerancia a fallos

Antes de detener ningún nodo, `SELECT COUNT(*) FROM pedidos` devolvió **10 000** filas contra el nodo 1 (conteo base). Tras detener el nodo 2 (`docker stop crdb-node2`) y confirmar mediante `docker compose ps -a` que su estado pasó a `Exited`, se reejecutó la misma consulta: el resultado se mantuvo en **10 000** filas, sin error ni degradación observable.

Explicación del quórum Raft 2/3: CockroachDB replica cada rango en un grupo Raft de n réplicas ($n = 3$). Una operación solo se confirma cuando la ha reconocido un quórum de $\lceil (n+1)/2 \rceil = 2$ réplicas. Con el nodo

2 caído, el rango de `pedidos` conservó 2 de sus 3 réplicas activas (nodos 1 y 3), cantidad suficiente para alcanzar el quórum.

Explicación de por qué SHOW RANGES no cambió tras 30 s: existe un parámetro de configuración —`server.time_until_store_dead`, con valor por defecto de 5 minutos— antes de que el clúster considere que un nodo está “muerto” y comience a generar réplicas de reemplazo. Dentro de la ventana de 30 segundos, el nodo 2 solo se encontraba “sospechoso”, no “muerto”; además, el rango seguía teniendo quórum (2/3). Por ambas razones, `SHOW RANGES` continuó reportando la misma configuración de tres réplicas `{1, 2, 3}` (Fig. 4).

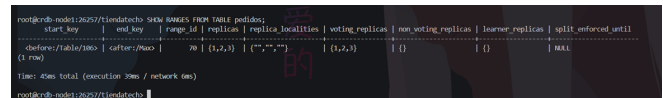


Figure 4: Salida de `SHOW RANGES FROM TABLE pedidos` durante la caída del nodo 2: la configuración de réplicas `{1, 2, 3}` no cambió dentro de la ventana de 30 s.

La Fig. 5 esquematiza el comportamiento Raft observado: mientras el nodo 2 permanece caído, el cliente SQL continúa dirigiendo sus solicitudes al *gateway* (nodo 1), que las enruta al líder vigente del rango; si dicho líder residía en el nodo 2, se produce una reelección resuelta entre el nodo 1 y el nodo 3.

Finalmente, se reinició el nodo 2 con `docker start crdb-node2` (Fig. 6). Se cronometró el tiempo de reintegración, desde la ejecución del comando hasta que `cockroach node status` reportó `is_available=true` e `is_live=true` en los tres nodos simultáneamente: el nodo se reintegró correctamente en **5.73 segundos**.

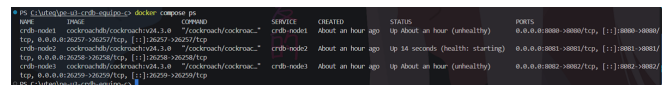


Figure 6: Reintegración del nodo 2 al clúster: los tres nodos vuelven a reportar `is_live=true` tras `docker start crdb-node2`.

4.2 Fragmentación lógica vs. rangos físicos

Corrección respecto de una interpretación previa del equipo: una versión anterior de este informe afirmaba que cada una de las cinco particiones lógicas de `pedidos` se almacenaba en un rango físico distinto del clúster. La evidencia recogida en la propia práctica no respalda esa afirmación.

`SHOW PARTITIONS FROM TABLE pedidos` (Fig. 2) confirmó la existencia de cinco particiones lógicas registradas en el catálogo. Sin embargo, al ejecutar `SHOW RANGES FROM TABLE pedidos` sobre la tabla completa, el resultado devolvió **una sola fila**: un único rango cuyo `start_key` y `end_key` abarcan la totalidad de la tabla, con `replicas = {1,2,3}`.

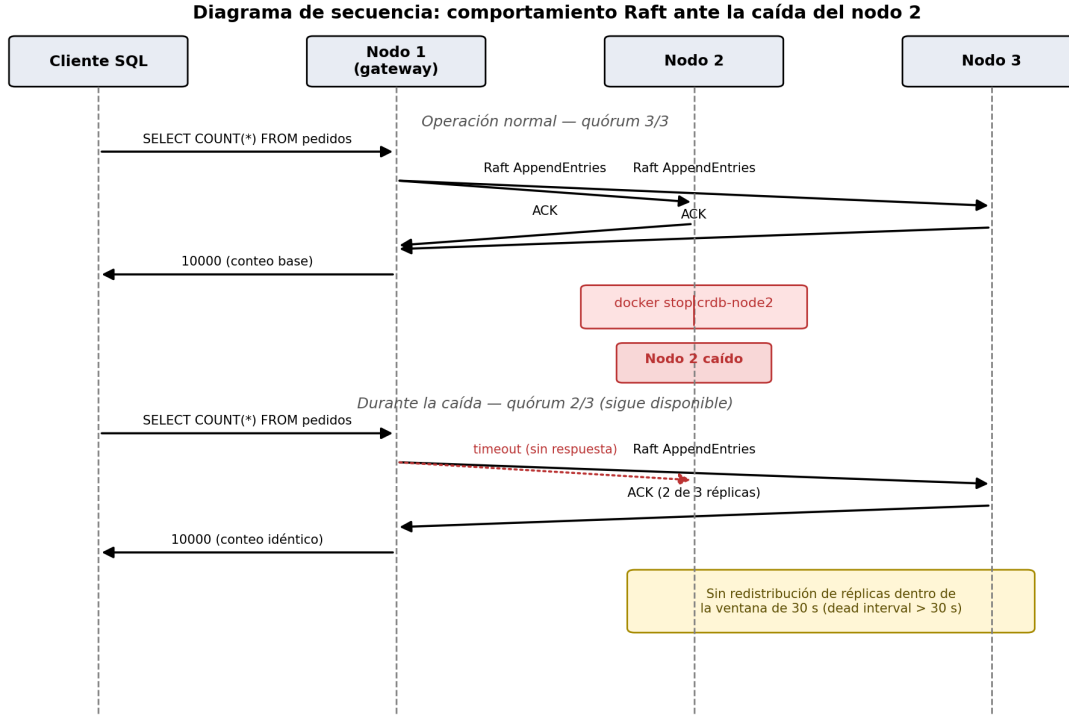


Figure 5: Diagrama de secuencia del comportamiento Raft ante la caída del nodo 2: el clúster mantiene disponibilidad de lectura con quórum 2/3.

Interpretación técnica correcta: PARTITION BY LIST crea unidades lógicas de configuración, pero no obliga a que cada partición resida en un rango físico separado. Un rango solo se subdivide cuando su tamaño supera 512 MiB o cuando se ejecuta explícitamente un comando de división. La tabla `pedidos`, con 10 000 filas, se mantuvo muy por debajo de ese umbral: las cinco particiones lógicas coexistieron dentro de un único rango físico, replicado tres veces mediante Raft.

Esto no invalida el objetivo pedagógico de la fragmentación horizontal —la definición formal de la Sección 2.2 y su verificación en el catálogo siguen siendo válidas—, pero sí corrige una inferencia errónea sobre la distribución física: a esta escala, la fragmentación lógica no implicó paralelismo de acceso entre rangos, y buena parte de la latencia observada corresponde a la coordinación de un único grupo Raft replicado tres veces.

4.3 Rendimiento: clúster vs. nodo único

La Tabla 3 presenta el tiempo promedio de ejecución (media de 5 repeticiones) de las cinco consultas, junto con el factor de comparación $F = t_{\text{nodo único}}/t_{\text{clúster}}$: $F < 1$ favorece al nodo único y $F > 1$ favorece al clúster.

Table 3: Tiempo promedio de ejecución (ms, $n = 5$) y factor de comparación entre nodo único y clúster

Consulta	Clúst.	N.ú.	F	Mejor
Q1 – <i>Join</i>	6.8	5.4	0.794	N. único
Q2 – GROUP BY	12.2	12.0	0.984	N. único
Q3 – Clave primaria	1.4	0.449	0.321	N. único
Q4 – Rango	12.8	14.4	1.125	Clúster
Q5 – Subconsulta	15.0	22.8	1.520	Clúster

Los resultados fueron **mixtos**: el nodo único presentó menor tiempo promedio en Q1, Q2 y Q3, mientras que el clúster fue más rápido en Q4 y Q5. Por tanto, no se puede afirmar que uno de los dos entornos superó al otro en todas las consultas; el rendimiento relativo dependió del tipo y del costo de procesamiento de cada operación.

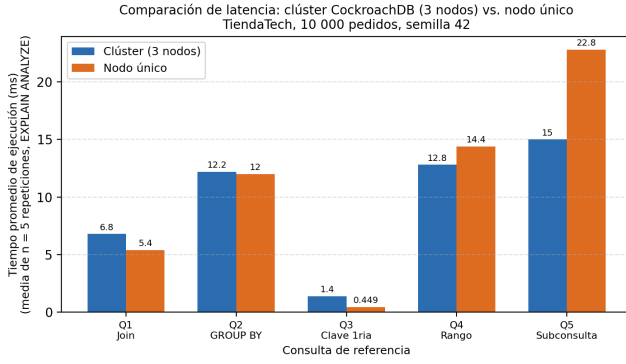


Figure 7: Gráfico comparativo de la latencia promedio (ms) por consulta, clúster de 3 nodos vs. nodo único, generado a partir de `evidencia/resultados.csv`.

Nota sobre reproducibilidad del gráfico: el gráfico anterior se generó con un script Python (`matplotlib`) que lee directamente los cinco pares de valores promedio de `evidencia/resultados.csv`; no contiene valores estimados ni inventados.

4.4 Análisis del plan de ejecución (EXPLAIN ANALYZE)

Más allá del tiempo total, el plan físico capturado por `EXPLAIN ANALYZE` explica por qué algunas consultas se comportaron distinto. La Fig. 8 muestra el plan real de Q1 en el clúster: el optimizador eligió un *lookup join* —para cada fila de `pedidos` escaneada, se realiza una búsqueda puntual contra `clientes@clientes_pkey`—, con un tiempo acumulado en KV de 24 ms y un uso máximo de memoria de 240 KiB.

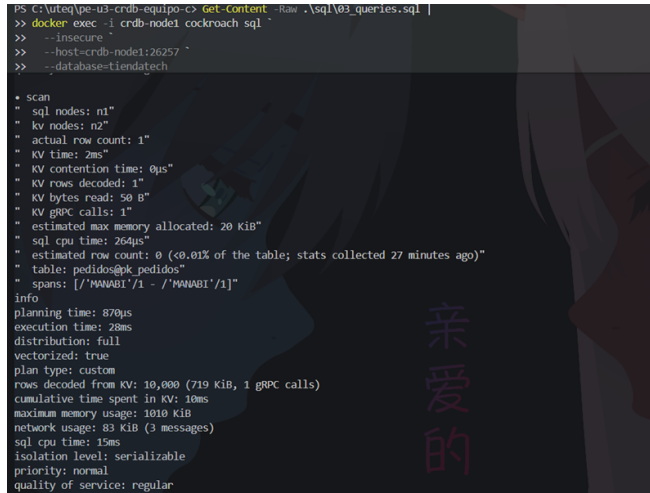


Figure 8: Plan de ejecución real de Q1 (`EXPLAIN ANALYZE`) en el clúster: *lookup join* entre `pedidos` y `clientes`.

Interpretación: un *lookup join* emite, por cada fila del lado externo, una solicitud KV independiente hacia

el nodo que posee la réplica líder de `clientes`; en un nodo único esas solicitudes se resuelven en memoria local, mientras que en el clúster cada una implica, en el peor caso, una llamada gRPC entre nodos, lo que explica una parte relevante de los 6.8 ms de Q1 en el clúster frente a los 5.4 ms del nodo único.

La Fig. 9 muestra el plan de Q3, donde el optimizador reportó `actual row count: 1`, confirmando un acceso directo por índice. Aun así, Q3 fue la consulta donde el nodo único obtuvo mayor ventaja relativa (factor 0.321): al tratarse de la operación de menor costo computacional posible, el tiempo total en el clúster quedó dominado casi por completo por el costo fijo de coordinación Raft. Este mismo razonamiento no se sostiene, sin embargo, para Q4 y Q5, donde el clúster fue más rápido; la Sección 4.5 profundiza en esta aparente contradicción.

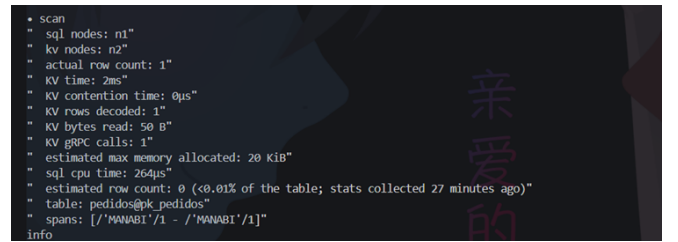


Figure 9: Plan de ejecución real de Q3 (`EXPLAIN ANALYZE`): búsqueda puntual por clave primaria compuesta, `actual row count = 1`.

4.5 Síntesis cuantitativa de resultados

Se calcula el promedio simple de los cinco factores F : $(0.794 + 0.984 + 0.321 + 1.125 + 1.520)/5 = 0.949$. Este resultado, cercano a 1, indica un rendimiento global similar entre ambos entornos, aunque oculta diferencias importantes entre consultas: el nodo único fue más rápido en tres de las cinco consultas (Q1, Q2, Q3), mientras que el clúster obtuvo mejores resultados en las otras dos (Q4, Q5).

La mayor ventaja del nodo único ocurrió en Q3, donde fue aproximadamente **3.12 veces más rápido** ($1/0.321$). La mayor ventaja del clúster ocurrió en Q5, donde fue **1.52 veces más rápido**. En Q2, ambos entornos presentaron tiempos prácticamente equivalentes (12.2 ms vs. 12.0 ms). Por tanto, no existió un ganador uniforme: el rendimiento dependió del tipo de consulta y de su costo de procesamiento.

5. Discusión

5.1 Contraste con trabajos relacionados

Los resultados de rendimiento se contrastan con cuatro trabajos académicos indexados, separando lo que reportan de la interpretación que se hace de ellos.

(a) Taft et al. (2020). Los autores documentan que CockroachDB prioriza deliberadamente la consisten-

cia serializable y la tolerancia a fallos sobre la latencia mínima de una única instancia [2]. *Interpretación:* nuestros datos son parcialmente coherentes: en las consultas de menor costo (Q1-Q3) el sobrecosto de coordinación fue visible, pero en Q4-Q5 el clúster fue más rápido, lo que sugiere que el costo de la replicación síncrona no domina cuando el procesamiento es suficientemente pesado.

(b) **Pina, Sá y Bernardino (2023)**. Los autores reportan que CockroachDB fue, de tres bases NewSQL evaluadas, la de mayores latencias, aunque con mejor escalabilidad [10]. *Interpretación:* nuestros resultados coincidieron parcialmente con este estudio. El nodo único fue más rápido en Q1, Q2 y Q3; sin embargo, el clúster obtuvo menores tiempos en Q4 y Q5. Esto indica que, con 10 000 pedidos, el costo de coordinación distribuida afectó principalmente a las consultas simples, mientras que las consultas de mayor procesamiento pudieron beneficiarse del entorno distribuido.

(c) **Li, Liu y Li (2022)**. Identifican que el diseño de líder fuerte de Raft se convierte en cuello de botella cuando crece la concurrencia [5]. *Interpretación:* aunque nuestro *benchmark* no generó concurrencia, el mecanismo que describen es consistente con la penalización observada en Q3 (nodo único $3.12\times$ más rápido), pero no explica por qué el clúster superó al nodo único en Q4 y Q5.

(d) **Assad, Meiklejohn, Miller y Krusche (2024)**. Describen una herramienta de inyección de fallos que soporta explícitamente CockroachDB [11]. *Interpretación:* nuestra prueba de tolerancia a fallos usó un método manual equivalente (`docker stop`) pero limitado a un único escenario; la Sección 5.3 retoma este trabajo.

Síntesis: los trabajos analizados explican el costo de coordinación, replicación y liderazgo de Raft en CockroachDB. Los resultados de esta práctica fueron mixtos: el nodo único presentó mejor rendimiento en tres consultas, mientras que el clúster fue más rápido en dos. La sobrecarga distribuida no afectó de manera uniforme a todas las operaciones, sino que dependió del tipo y costo de cada consulta.

5.2 Amenazas a la validez

1. **Validez interna — entorno compartido de recursos:** ambos entornos se ejecutaron en la misma máquina host, compitiendo por CPU y E/S de disco.
2. **Validez interna — ausencia de calentamiento y de estadística por repetición:** el script solo conservó el promedio, no la dispersión entre repeticiones.
3. **Validez externa — escala del dataset:** 10 000 pedidos es un volumen reducido; los resultados mixtos no deben extrapolarse a escenarios de millones de filas.
4. **Validez externa — ausencia de concurrencia:**

no se evaluó el comportamiento bajo múltiples conexiones concurrentes.

5. **Validez de constructo:** el tiempo de EXPLAIN ANALYZE excluye la latencia de red entre la aplicación Python y el clúster.

5.3 Trabajo futuro

1. **Instrumentar el benchmark para conservar datos crudos por repetición,** permitiendo calcular desviación estándar, mediana y percentiles.
2. **Adoptar una herramienta de inyección de fallos automatizada** [11], ampliando la prueba más allá de la caída de un nodo completo.
3. **Repetir el benchmark a mayor escala y con concurrencia,** para verificar si la proporción de consultas favorables al clúster aumenta con el volumen [10], y forzar la división en múltiples rangos físicos.
4. **Medir la latencia end-to-end percibida por el cliente,** complementando EXPLAIN ANALYZE con mediciones del lado de la aplicación.

6. Conclusiones

RQ1. Se desplegó un clúster CockroachDB v24.3.0 de tres nodos en contenedores Docker, y se implementó el esquema de cuatro tablas de TiendaTech con fragmentación lógica PARTITION BY LIST en cinco particiones, verificadas mediante SHOW PARTITIONS. Se cargaron 10 000 pedidos y 19 884 detalles de forma reproducible (semilla 42).

RQ2. El clúster preservó la disponibilidad y la consistencia de lectura durante la caída del nodo 2. La consulta SELECT COUNT(*) FROM pedidos devolvió 10 000 registros antes y después de detener el nodo, sin presentar errores, confirmando el quórum Raft de 2 de 3 réplicas. Tras docker start crdb-node2, el nodo se reintegró correctamente en un tiempo medido de **5.73 segundos**.

RQ3. Los resultados de rendimiento fueron mixtos. El nodo único presentó menor tiempo promedio en Q1, Q2 y Q3, mientras que el clúster fue más rápido en Q4 y Q5. La mayor ventaja del nodo único ocurrió en Q3 ($3.12\times$); la mayor ventaja del clúster ocurrió en Q5 ($1.52\times$). El promedio de los cinco factores fue **0.949**, cercano a 1, indicando un rendimiento global semejante, aunque este promedio no representó por completo las diferencias individuales.

La tabla pedidos permaneció en un único rango físico replicado en los tres nodos; por tanto, las diferencias de rendimiento no estuvieron relacionadas con la distribución entre varios rangos físicos, sino con el procesamiento de cada consulta y la coordinación distribuida realizada por CockroachDB.

Se concluyó que el principal beneficio del clúster para TiendaTech fue la continuidad del servicio ante la caída de un nodo. No se encontró un entorno uniformemente más rápido para todas las consultas, por lo que la selección entre un clúster y un nodo único debe considerar tanto la disponibilidad requerida como los patrones reales de acceso del sistema.

7. Anexos

A.1 Consistencia de CockroachDB ante fallo de nodo (algoritmo Raft)

CockroachDB divide cada tabla en rangos y replica cada rango en un grupo Raft independiente —en la práctica realizada, la tabla `pedidos` completa correspondió a un único rango (Sección 4.2)—. Una réplica es elegida líder mediante elecciones basadas en *heartbeats*; una entrada del *log* se considera comprometida solo cuando la ha persistido un quórum de $\lceil (n+1)/2 \rceil$ réplicas. Si el líder falla, los seguidores restantes detectan la ausencia de *heartbeats* y convocan una nueva elección.

A.2 CockroachDB (Serializable) vs. Apache Cassandra (consistencia eventual)

CockroachDB ofrece `SERIALIZABLE`; Cassandra ofrece consistencia ajustable por cuórum (`ONE`, `QUORUM`, `ALL`). Para TiendaTech, cuyo dominio exige integridad referencial estricta, se elegiría CockroachDB: perder consistencia en una venta tiene mayor costo de negocio que la penalización de latencia medida en esta práctica.

A.3 Fragmentación horizontal explícita con `PARTITION BY RANGE`

Una estrategia complementaria a `LIST` sería `PARTITION BY RANGE` sobre `fecha_pedido`. Al igual que con la partición por lista, definir particiones lógicas por rango de fecha no garantizaría por sí sola una distribución física en múltiples rangos si el volumen total permanece bajo 512 MiB.

A.4 Atomicidad multi-tabla en distintos nodos: 2PC interno

CockroachDB garantiza atomicidad mediante una variante interna del compromiso en dos fases: el nodo *gateway* designa un registro de transacción y coordina un `Prepare` implícito escribiendo valores *intent*; solo cuando todos se han escrito exitosamente se marca la transacción como `COMMITTED`.

A.5 Preguntas de reflexión adicionales

A 10 000 000 de pedidos, la tabla superaría el umbral de 512 MiB, forzando su división en decenas de rangos; en

ese escenario, la fragmentación lógica por provincia sí podría traducirse en rangos físicos diferenciados si se combina con una directiva explícita de pre-división (`ALTER TABLE ... SPLIT AT`).

Referencias

- [1] M. Tamer Özsu and Patrick Valduriez. *Principles of Distributed Database Systems*. Springer, Cham, Switzerland, 4th edition, 2020. Obra de referencia clásica del área, indispensable para la notación formal de fragmentación y replicación; publicación indexada. doi: 10.1007/978-3-030-26253-2.
- [2] Rebecca Taft, Irfan Sharif, Andrei Matei, Nathan VanBenschoten, Jordan Lewis, Tobias Grieger, Kai Niemi, Andy Woods, Anne Birzin, Raphael Poss, Paul Bardea, Amruta Ranade, Ben Darnell, Bram Gruneir, Justin Jaffray, Lucy Zhang, and Peter Mattis. CockroachDB: The resilient geo-distributed SQL database. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 1493–1509, Portland, OR, USA, 2020. doi: 10.1145/3318464.3386134.
- [3] Jim Gray and Leslie Lamport. Consensus on transaction commit. *ACM Transactions on Database Systems*, 31(1):133–160, March 2006. doi: 10.1145/1132863.1132867.
- [4] Michael J. Fischer, Nancy A. Lynch, and Michael S. Paterson. Impossibility of distributed consensus with one faulty process. *Journal of the ACM*, 32(2):374–382, April 1985. doi: 10.1145/3149.214121.
- [5] Hongyan Li, Zheng Liu, and Yun Li. An improved Raft consensus algorithm based on asynchronous batch processing. In *Proceedings of the 2021 International Conference on Wireless Communications, Networking and Applications (WCNA 2021), Lecture Notes in Electrical Engineering*, Singapore, 2022. Springer. doi: 10.1007/978-981-19-2456-9_44.
- [6] Eric Brewer. CAP twelve years later: How the “rules” have changed. *IEEE Computer*, 45(2):23–29, February 2012. doi: 10.1109/MC.2012.37.
- [7] Daniel J. Abadi. Consistency tradeoffs in modern distributed database system design. *IEEE Computer*, 45(2):37–42, February 2012. doi: 10.1109/MC.2012.33.
- [8] James C. Corbett et al. Spanner: Google’s globally-distributed database. *ACM Transactions on Computer Systems*, 31(3):1–22, August 2013. Artículo 8. doi: 10.1145/2491245.

- [9] Dongxu Huang et al. TiDB: A Raft-based HTAP database. *Proceedings of the VLDB Endowment*, 13(12):3072–3084, 2020. doi: 10.14778/3415478.3415535.
- [10] Eduardo Pina, Filipe Sá, and Jorge Bernardino. NewSQL databases assessment: CockroachDB, MariaDB xpack, and VoltDB. *Future Internet*, 15(1), 2023. Artículo 10. doi: 10.3390/fi15010010.
- [11] Michael Assad, Christopher S. Meiklejohn, Heather Miller, and Stephan Krusche. Can my microservice tolerate an unreliable database? Resilience testing with fault injection and visualization. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion '24)*, Lisbon, Portugal, 2024. doi: 10.1145/3639478.3640021.